SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO3: *Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)*

Assessment Tool Weightage: 50%

Time Duration: 1 Hour

QUESTIONS: 4 **Total Marks:40**

Annexure A

CASE STUDY

Code of Conduct:

I K. Pujitha (192524149) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Pujitha

Annexure A

CASE STUDY

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

I. If a patient has Fever AND Cough → Possible Flu

Ii. If a patient has Fever AND Rash → Possible Measles

Iii. If a patient has Cough AND Breathlessness → Possible Pneumonia

Iv. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

I. ∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) [Rule 1]
II. ∀x: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) [Rule 2]
III. ∀x ∀y: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x) [Rule 3]
IV. Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance)

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.

(b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

1) P1: SoilDry → IrrigationNeeded
2) P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod
3) P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk
4) Facts: SoilDry = True, CropWheat = True

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication → move ¬ inward → distribute).

(b) Using Resolution Refutation, prove the theorem 'ApplyDripMethod'. Negate the goal, combine with the KB in CNF, and derive the empty clause □. Draw the resolution proof tree.

(c) If the fact 'SoilDry' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'CropAtRisk' still follow? Apply appropriate reasoning patterns to justify.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

1) Stench [1,2] → Wumpus is adjacent to cell [1,2]
2) Breeze [1,1] → Pit is adjacent to cell [1,1]
3) Glitter [x,y] → Gold is at [x,y]
4) ¬Wumpus [x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

(c) Evaluate whether the path [1,1] → [1,2] → [2,2] is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1 — Smart Medical Diagnosis System

Given Knowledge Base

The hospital's rule-based system contains:

1. If a patient has Fever AND Cough → Possible Flu
2. If a patient has Fever AND Rash → Possible Measles
3. If a patient has Cough AND Breathlessness → Possible Pneumonia
4. Patient A has:
    - Fever = True
    - Cough = True
    - Breathlessness = True

(a) Representation using Propositional Logic

Step 1: Define the symbols

For **Patient A**, let:

Symbol	Meaning
F	Patient A has Fever
C	Patient A has Cough
B	Patient A has Breathlessness
R	Patient A has Rash
Flu	Patient A has Possible Flu
M	Patient A has Possible Measles
P	Patient A has Possible Pneumonia

Step 2: Represent the rules

Rule I: Fever AND Cough → Possible Flu

$F \land C \rightarrow Flu$

Rule II: Fever AND Rash → Possible Measles

$F \land R \rightarrow M$

Rule III: Cough AND Breathlessness → Possible Pneumonia

$C \land B \rightarrow P$

Step 3: Represent the facts

For Patient A:

F=True C=True B=True

There is **no fact stating that Patient A has Rash**, so:

R=False/Unknown

Complete Knowledge Base

KB={F∧C→Flu,F∧R→M,C∧B→P,F,C,B}

Symbols used

- **∧** = AND
- → = implies / leads to
- **True** = the fact is known to be true
- **False/Unknown** = the fact is not established as true

(b) Forward Chaining

What is Forward Chaining?

Forward chaining is **a** data-driven inference method.

It starts with the known facts and repeatedly applies rules whose conditions are satisfied to derive new facts.

Initial Facts

F=True C=True B=True

Step 1: Apply Rule I

Rule I:

F∧C→Flu

We know:

F=True,C=True

Therefore, both conditions are satisfied.

So:

Flu=True

Inference: Patient A has Possible Flu.

Step 2: Apply Rule II

Rule II:

$F \wedge R \rightarrow M$

We know:

$F = True$

But Rash (R) is not given.

Therefore, the rule cannot be fired.

Measles cannot be derived

Step 3: Apply Rule III

Rule III:

$C \wedge B \rightarrow P$

We know:

$C = True, B = True$

Therefore, both conditions are satisfied.

So:

$Pneumonia = True$

Inference: Patient A has Possible Pneumonia.

Forward Chaining Table

Step	Rule	Conditions	New Fact
Initial	—	F, C, B are true	Fever, Cough, Breathlessness
1	Rule I	F ∧ C	**Flu**
2	Rule II	F ∧ R	Cannot fire; Rash unavailable
3	Rule III	C ∧ B	**Pneumonia**

Final Derived Facts

$Flu = True$ $Pneumonia = True$

Measles cannot be derived because **Rash is not known**.

Therefore:

Patient A is diagnosed with Possible Flu and Possible Pneumonia based on the given rules and facts.

Note that these rules say **"Possible"** diagnoses, so this is a preliminary rule-based inference, not a definitive medical diagnosis.

(c) Backward Chaining for Pneumonia

What is Backward Chaining?

Backward chaining is a **goal-driven inference method**.

It starts with the desired goal and works **backward** to determine which rules can prove that goal.

Given Goal

P

where **P = Patient A has Possible Pneumonia**.

Step 1: Start with the Goal

P

We search the knowledge base for a rule whose conclusion is **P**.

The matching rule is:

$C \wedge B \rightarrow P$

Therefore, to prove **P**, we need to prove:

$C \wedge B$

Step 2: Reduce the Goal

The goal:

P

is reduced to two sub-goals:

C

and

B

Step 3: Verify Sub-goal C

$C = Cough$

Cough is already given as a fact:

$C = True$

Therefore, this sub-goal is satisfied.

Step 4: Verify Sub-goal B

B=Breathlessness

Breathlessness is also given as a fact:

B=True

Therefore, this sub-goal is satisfied.

Goal Reduction Tree

```
          GOAL
  Patient A has Pneumonia (P)
            |
    Rule III: C ∧ B → P
            |
     ┌──────┴───────┐
     ↓        ↓
    Cough (C)  Breathlessness (B)
     |        |
    FACT ✓     FACT ✓
     |        |
     True      True
```

Since both sub-goals are satisfied, the original goal is proven.

$C \land B \Rightarrow P$

Therefore:

Patient A has Possible Pneumonia

Case Study 2: Given Knowledge Base

Rule 1

$\forall x[Vehicle(x) \land EmergencyType(x) \rightarrow ClearPath(x)]$

Rule 2

$\forall x[ClearPath(x) \rightarrow GreenSignal(x) \land \neg RedSignal(x)]$

Rule 3

$\forall x \forall y[Vehicle(x) \land Behind(x,y) \land GreenSignal(y) \rightarrow Proceed(x)]$

Facts

Vehicle(Ambulance) EmergencyType(Ambulance) Vehicle(CarA) Behind(CarA,Ambulance)

(a) Unification of Rule 1 with Given Facts

What is Unification?

Unification is the process of finding substitutions for variables so that two logical expressions become identical.

The substitution set is represented by:

θ

The **MGU (Most General Unifier)** is the most general substitution that makes the expressions match.

Rule 1

$Vehicle(x) \land EmergencyType(x) \rightarrow ClearPath(x)$

For applying the rule, we need to match its premises with the facts.

Step 1: Match Vehicle(x)

Given fact:

Vehicle(Ambulance)

Rule premise:

Vehicle(x)

To make them identical:

$x = Ambulance$

Therefore,

$\theta 1 = \{x/Ambulance\}$

Step 2: Match EmergencyType(x)

Rule premise:

EmergencyType(x)

Given fact:

EmergencyType(Ambulance)

Using:

$\theta 1=\{x/Ambulance\}$

we obtain:

EmergencyType(Ambulance)

which matches the fact exactly.

Therefore:

$\theta 2=\{x/Ambulance\}$

MGU

Hence, the complete substitution is:

$\theta=\{x/Ambulance\}$

Applying this substitution to Rule 1:

$Vehicle(Ambulance) \land EmergencyType(Ambulance) \rightarrow ClearPath(Ambulance)$

Both premises are true, so we derive:

ClearPath(Ambulance)

Unification Summary

Expression	Given Fact	Substitution
Vehicle(x)	Vehicle(Ambulance)	x/Ambulance
EmergencyType(x)	EmergencyType(Ambulance)	x/Ambulance
ClearPath(x)	—	ClearPath(Ambulance)

Therefore:

$\theta=\{x/Ambulance\}$

(b) Resolution Proof of Proceed(CarA)

Goal

We need to prove:

Proceed(CarA)

In **Resolution**, we prove a goal by adding its negation to the knowledge base and deriving a contradiction.

So we add:

$\neg Proceed(CarA)$

If we derive the empty clause:

□

then the goal is proven.

Step 1: Convert Rules to CNF

Rule 1

Original:

$Vehicle(x) \wedge EmergencyType(x) \rightarrow ClearPath(x)$

Using:

$A \wedge B \rightarrow C \equiv \neg A \vee \neg B \vee C$

Therefore:

$\neg Vehicle(x) \vee \neg EmergencyType(x) \vee ClearPath(x)$

Call this **C1**.

Rule 2

Original:

$ClearPath(x) \rightarrow GreenSignal(x) \wedge \neg RedSignal(x)$

This gives two CNF clauses:

$\neg ClearPath(x) \vee GreenSignal(x)$

C2

and

$\neg ClearPath(x) \vee \neg RedSignal(x)$

C3

For proving Proceed(CarA), C3 is not required.

Rule 3

Original:

$Vehicle(x) \land Behind(x,y) \land GreenSignal(y) \rightarrow Proceed(x)$

Convert to CNF:

$\neg Vehicle(x) \lor \neg Behind(x,y) \lor \neg GreenSignal(y) \lor Proceed(x)$

Call this **C4**.

Step 2: Ground the Relevant Clauses

From the facts:

Vehicle(Ambulance) EmergencyType(Ambulance)

Using C1 with:

$\theta = \{x/Ambulance\}$

we get:

$\neg Vehicle(Ambulance) \lor \neg EmergencyType(Ambulance) \lor ClearPath(Ambulance)$

Resolve with:

Vehicle(Ambulance)

to obtain:

$\neg EmergencyType(Ambulance) \lor ClearPath(Ambulance)$

Resolve with:

EmergencyType(Ambulance)

Therefore:

ClearPath(Ambulance)

Step 3: Derive Green Signal

C2:

$\neg ClearPath(x) \lor GreenSignal(x)$

Substitute:

x=Ambulance

giving:

$\neg ClearPath(Ambulance) \lor GreenSignal(Ambulance)$

We already have:

$ClearPath(Ambulance)$

Resolving:

$GreenSignal(Ambulance)$

Step 4: Use Rule 3

C4:

$\neg Vehicle(x) \lor \neg Behind(x,y) \lor \neg GreenSignal(y) \lor Proceed(x)$

For CarA and Ambulance, substitute:

$\theta = \{x/CarA, y/Ambulance\}$

So:

$\neg Vehicle(CarA) \lor \neg Behind(CarA,Ambulance) \lor \neg GreenSignal(Ambulance) \lor Proceed(CarA)$

We have the facts:

$Vehicle(CarA)$ $Behind(CarA,Ambulance)$

and derived fact:

$GreenSignal(Ambulance)$

Resolving with these positive facts successively gives:

$Proceed(CarA)$

Resolution Refutation

To formally prove the goal, add:

$\neg Proceed(CarA)$

Now we have:

$Proceed(CarA)$

and

¬Proceed(CarA)

Resolution gives:

□

The empty clause represents a **contradiction**.

Therefore:

KB⊨Proceed(CarA)

Hence:

Proceed(CarA) is logically entailed by the knowledge base.

Complete Resolution Flow

```
Vehicle(Ambulance)
        +
EmergencyType(Ambulance)
        |
      Rule 1
        ↓
ClearPath(Ambulance)
        |
      Rule 2
        ↓
GreenSignal(Ambulance)
        |
        +------------------+
        |                  |
Vehicle(CarA)      Behind(CarA,Ambulance)
        |                  |
        +---------+---------+
                  |
              Rule 3
                  ↓
            Proceed(CarA)
                  |
                  +  ¬Proceed(CarA)
                  ↓
             CONTRADICTION
                  ↓
                  □
```

(c) Can the KB Handle Two Simultaneous Emergency Vehicles?

Answer: Yes, partially — but it needs additional information/rules for proper traffic management.

The current FOL rules use:

$\forall x$

and therefore **Rule 1 and Rule 2 can apply independently to any number of vehicles**.

For example, suppose there are two ambulances:

Ambulance1

and

Ambulance2

with facts:

Vehicle(Ambulance1) EmergencyType(Ambulance1) Vehicle(Ambulance2) EmergencyType(Ambulance2)

Rule 1 can independently derive:

ClearPath(Ambulance1)

and

ClearPath(Ambulance2)

Then Rule 2 can derive:

GreenSignal(Ambulance1)

and

GreenSignal(Ambulance2)

So the **basic inference mechanism supports multiple emergency vehicles**.

But there is a problem

The current knowledge base does **not specify what happens when two emergency vehicles require the same path or intersection at the same time**.

For example:

```
Ambulance1 ──────────→ Intersection
            ↑
Ambulance2 ────────────────┘
```

Both could potentially receive:

GreenSignal

There is no rule deciding:

- Which emergency vehicle gets priority?
- Whether both routes can safely be cleared.
- Whether conflicting paths should be avoided.
- What happens if the vehicles approach the same intersection from different directions.

Additional Facts Needed

We could add facts such as:

At(Ambulance1,Intersection1) At(Ambulance2,Intersection1)
Route(Ambulance1,Route1) Route(Ambulance2,Route2)

and perhaps:

EmergencyLevel(Ambulance1,High) EmergencyLevel(Ambulance2,Medium)

These facts allow the system to distinguish between the two emergency vehicles and their routes.

Additional Rule 1: Emergency Priority

A priority rule could be introduced:

$$\forall x \forall y [EmergencyType(x) \land EmergencyType(y) \land HigherPriority(x,y) \rightarrow PriorityVehicle(x)]$$

This allows the system to determine which emergency vehicle should receive priority.

Additional Rule 2: Avoid Conflicting Routes

A rule can specify that two vehicles should not proceed simultaneously when their paths conflict:

$$\forall x \forall y [Conflict(x,y) \rightarrow \neg Proceed(x) \lor \neg Proceed(y)]$$

This prevents simultaneous movement when the routes are unsafe.

Additional Rule 3: Give Green Signal to the Priority Vehicle

For example:

$\forall x[PriorityVehicle(x) \rightarrow GreenSignal(x)]$

This gives the higher-priority emergency vehicle the green signal.

Logical Justification

The original rules are generalized using variables, so they can reason about multiple vehicles:

$\forall x$

However, they do not contain predicates such as:

Priority(x) Conflict(x,y) Same Intersection(x,y)

Therefore, the current KB can identify and clear paths for multiple emergency vehicles, but it cannot adequately resolve conflicts between simultaneous emergency vehicles**.**

Case Study 3 — Agricultural Expert Reasoning System

We are given:

- **P1:** SoilDry → IrrigationNeeded
- **P2:** IrrigationNeeded ∧ CropWheat → ApplyDripMethod
- **P3:** ¬ApplyDripMethod ∧ CropWheat → CropAtRisk
- **Facts:** SoilDry = True, CropWheat = True

Let:

- S = SoilDry
- I = IrrigationNeeded
- W = CropWheat
- D = ApplyDripMethod
- R = CropAtRisk

(a) Convert P1, P2, P3 and Facts into CNF

What is CNF?

Conjunctive Normal Form (CNF) is a conjunction (**AND**) of clauses, where each clause contains literals connected by **OR**.

Example:

$(A \lor B) \land (C \lor D)$

P1: SoilDry → IrrigationNeeded

Original:

$S \rightarrow I$

Step 1: Eliminate implication

Use:

$A \rightarrow B \equiv \neg A \lor B$

Therefore:

$S \rightarrow I$

becomes:

$\neg S \lor I$

P1 in CNF:

$\neg SoilDry \lor IrrigationNeeded$

P2: Irrigation Needed ∧ CropWheat → ApplyDripMethod

Original:

$(I \land W) \rightarrow D$

Step 1: Eliminate implication

$A \rightarrow B \equiv \neg A \lor B$

Therefore:

$\neg (I \land W) \lor D$

Step 2: Move negation inward

Using De Morgan's law:

$\neg (A \land B) \equiv \neg A \lor \neg B$

Therefore:

$\neg I \lor \neg W \lor D$

No distribution is needed because this is already one clause.

P2 in CNF:

$\neg I \vee \neg W \vee D$

P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk

Original:

$(\neg D \wedge W) \rightarrow R$

Step 1: Eliminate implication

$\neg(\neg D \wedge W) \vee R$

Step 2: Move negation inward

Using De Morgan's law:

$\neg(\neg D \wedge W) \equiv D \vee \neg W$

Therefore:

$D \vee \neg W \vee R$

P3 in CNF:

$D \vee \neg W \vee R$

Again, distribution is not required because the expression is already a single clause.

Facts in CNF

Given:

$SoilDry = True$

Therefore:

S

Given:

$CropWheat = True$

Therefore:

W

Both are already single-literal CNF clauses.

Complete CNF Knowledge Base

Therefore:

$(\neg S \vee I) \wedge (\neg I \vee \neg W \vee D) \wedge (D \vee \neg W \vee R) \wedge S \wedge W$

CNF Clause List

Clause	CNF
C1	$\neg S \vee I$
C2	$\neg I \vee \neg W \vee D$
C3	$D \vee \neg W \vee R$
C4	S
C5	W

(b) Resolution Refutation to Prove ApplyDripMethod

Goal

We need to prove:

D

That is:

ApplyDripMethod is true.

Resolution Refutation Method

To prove D:

1. Convert KB into CNF.
2. Negate the goal.
3. Add the negated goal to the KB.
4. Apply resolution.
5. If we derive the empty clause □, the goal is proven.

Step 1: Negate the Goal

Goal:

D

Negated goal:

¬D

Add it to the CNF clauses.

So we now have:

- C1: ¬S∨I
- C2: ¬I∨¬W∨D
- C3: D∨¬W∨R
- C4: S
- C5: W
- C6: ¬D

Step 2: Resolve C1 and C4

C1:

¬S∨I

C4:

S

The complementary literals are:

¬S,S

Resolving them gives:

I

So:

IrrigationNeeded

Step 3: Resolve C2 and C5

C2:

¬I∨¬W∨D

C5:

W

Resolve W with ¬W:

¬I∨D

Therefore:

$\neg I \lor D$

Step 4: Resolve with IrrigationNeeded

We have:

I

and:

$\neg I \lor D$

Resolve I and ¬I:

D

Therefore:

ApplyDripMethod

Step 5: Resolve with the Negated Goal

We added:

$\neg D$

We have derived:

D

Resolve:

D

with:

$\neg D$

Therefore:

$\square$

The empty clause represents a contradiction.

Hence:

$KB \vDash ApplyDripMethod$

Resolution Proof Tree

```
                 KB + ¬D
                  |
    ---------------------------------
    |                       |
  C1: ¬S ∨ I                   C4: S
    |                       |
    -------- Resolution -------------
                 |
                 I
                 |
    C2: ¬I ∨ ¬W ∨ D      C5: W
           |          |
           ----- Resolution
                  |
               ¬I ∨ D
                  |
                  I
                  |
           ----- Resolution
                  |
                  D
                  |
                 ¬D
           (Negated Goal)
                  |
           ----- Resolution
                  |
                  □
```

Therefore:

∴ApplyDripMethod is proved.

(c) Remove the Fact Soil Dry

Now suppose:

SoilDry

is **removed** from the KB.

Originally we had:

S

But now S is no longer available.

The remaining facts are:

W

where W=CropWheat.

Step 1: Can we derive IrrigationNeeded?

P1 is:

$S \rightarrow I$

CNF:

$\neg S \lor I$

To derive I, we need:

S

But S has been removed.

Therefore:

IrrigationNeeded cannot be derived

Step 2: Can we derive ApplyDripMethod?

P2 requires:

$I \land W$

We have:

W=True

but we do **not** have:

I

Therefore:

ApplyDripMethod cannot be derived

Step 3: Can we derive CropAtRisk?

P3 is:

$\neg D \land W \rightarrow R$

To derive R, we need:

¬D

and:

W

We still have:

W=True

However, **there is no fact or rule that derives ¬D**.

Importantly, the inability to prove D does **not** mean that ¬D is true.

This is an important logical principle:

Not proving something is not the same as proving its negation.

Therefore:

CropAtRisk cannot be derived

Step-by-Step Comparison

Situation	Original KB	After removing SoilDry
SoilDry S	True	Not available
CropWheat W	True	True
IrrigationNeeded I	**Derived**	Cannot derive
ApplyDripMethod D	**Derived**	Cannot derive
¬ApplyDripMethod	Not given	Still not given
CropAtRisk R	Cannot derive	Cannot derive

Important observation

Even in the **original KB**, CropAtRisk does **not** follow.

Why?

Because P3 requires:

¬ApplyDripMethod

But the original KB actually proves:

ApplyDripMethod

It does **not** prove:

¬ApplyDripMethod

Therefore, CropAtRisk is not logically entailed.

After removing SoilDry, we lose the chain:

$SoilDry \rightarrow IrrigationNeeded \rightarrow ApplyDripMethod$

But we still cannot conclude:

$\neg ApplyDripMethod$

So we still cannot conclude:

$CropAtRisk$

Case Study 4 — Wumpus World Knowledge Agent

We have the following percepts:

- **Stench at [1,2]**
- **Breeze at [1,1]**
- **Glitter at [2,2]**

Rules:

1. $Stench[1,2] \rightarrow WumpusAdjacent[1,2]$
2. $Breeze[1,1] \rightarrow PitAdjacent[1,1]$
3. $Glitter[x,y] \rightarrow Gold[x,y]$
4. $\neg Wumpus[x,y] \wedge \neg Pit[x,y] \rightarrow Safe[x,y]$

(a) Construct the Belief State and Apply Modus Ponens

1. Initial Percepts

The agent's initial belief state contains:

$Stench[1,2]$ $Breeze[1,1]$ $Glitter[2,2]$

These are the facts directly perceived by the agent.

2. Apply Modus Ponens

Rule 1

$Stench[1,2] \rightarrow WumpusAdjacent[1,2]$

We know:

$Stench[1,2]$

Therefore, by **Modus Ponens**:

$WumpusAdjacent[1,2]$

Rule 2

$Breeze[1,1] \rightarrow PitAdjacent[1,1]$

We know:

$Breeze[1,1]$

Therefore:

$PitAdjacent[1,1]$

Rule 3

$Glitter[x,y] \rightarrow Gold[x,y]$

We have:

$Glitter[2,2]$

Substitute:

$x=2, y=2$

Therefore:

$Gold[2,2]$

Rule 4

$\neg Wumpus[x,y] \land \neg Pit[x,y] \rightarrow Safe[x,y]$

To use this rule, we need both:

$\neg Wumpus[x,y]$

and

$\neg Pit[x,y]$

But these facts are **not provided**.

Therefore, Rule 4 cannot currently be applied.

Belief State After Inference

Percept / Fact	**Derived Knowledge**
Stench[1,2]	WumpusAdjacent[1,2]
Breeze[1,1]	PitAdjacent[1,1]
Glitter[2,2]	Gold[2,2]
—	Safe[x,y] cannot yet be derived

Final derived conclusions

WumpusAdjacent[1,2] PitAdjacent[1,1] Gold[2,2]

No specific Safe[x,y] fact can be derived from the given information.

(b) Forward Chaining to Identify Possible Wumpus and Pit Locations

Important Wumpus World assumption

In the standard Wumpus World, a **stench in a cell means the Wumpus is in one of the cells adjacent to it**.

Similarly, a **breeze means a pit is in one of the adjacent cells**.

For a grid, the adjacent cells are normally **up, down, left, and right**.

Step 1: Wumpus Location

We have:

Stench[1,2]

Therefore:

WumpusAdjacent[1,2]

The cells adjacent to [1,2] are:

[1,1],[1,3],[2,2]

assuming these cells exist in the grid.

Therefore, the Wumpus could be in:

$[1,1] \lor [1,3] \lor [2,2]$

So the current information does **not identify one exact Wumpus location**.

Step 2: Pit Location

We have:

Breeze[1,1]

Therefore:

PitAdjacent[1,1]

The adjacent cells to [1,1] are:

[1,2],[2,1]

Therefore, the possible pit locations are:

$[1,2] \lor [2,1]$

Step 3: Gold Location

We have:

Glitter[2,2]

Using Rule 3:

$\text{Glitter}[2,2] \rightarrow \text{Gold}[2,2]$

Therefore:

Gold[2,2]

Forward-Chaining Table

Step	Known Fact	Rule Applied	New Knowledge
1	Stench[1,2]	Rule 1	WumpusAdjacent[1,2]
2	Breeze[1,1]	Rule 2	PitAdjacent[1,1]
3	Glitter[2,2]	Rule 3	Gold[2,2]
4	WumpusAdjacent[1,2]	Adjacency	Wumpus ∈ [1,1],[1,3],[2,2]
5	PitAdjacent[1,1]	Adjacency	Pit ∈ [1,2],[2,1]

Possible locations

Wumpus: [1,1],[1,3],[2,2] Pit: [1,2],[2,1] Gold: [2,2]

(c) Is the Path [1,1] → [1,2] → [2,2] Safe?

The proposed path is:

[1,1]→[1,2]→[2,2]

We need to examine **each cell**.

Cell [1,1]

The agent perceives:

Breeze[1,1]

This tells us that a pit is adjacent to [1,1].

Possible pit locations are:

[1,2]or[2,1]

Therefore, the agent **cannot establish that [1,1] is completely safe** from the given rules.

Cell [1,2]

This cell has:

Stench[1,2]

This means the Wumpus is potentially in one of its adjacent cells:

[1,1],[1,3],[2,2]

But the stench **does not mean the Wumpus is actually in [1,2]**.

However, from the given KB we also know that a pit could be at:

[1,2]

because of the breeze at [1,1].

Therefore, the proposed path enters a cell that **could contain a pit**.

So:

[1,2] is not proven safe

Cell [2,2]

We know:

Gold[2,2]

However, the glitter only tells us that gold is present.

It does **not** prove that [2,2] is safe.

Furthermore, [2,2] is adjacent to [1,2], where there is a stench. Thus, under the standard adjacency interpretation, [2,2] is also a possible Wumpus location.

Therefore:

[2,2] is not proven safe

Safety Analysis

The safety rule is:

$\neg Wumpus[x,y] \land \neg Pit[x,y] \rightarrow Safe[x,y]$

To prove that a cell is safe, we need to establish **both**:

$\neg Wumpus[x,y]$

and

$\neg Pit[x,y]$

But the current KB does not provide these negative facts for the cells along the proposed path.

Therefore, we cannot logically prove:

Safe[1,1] Safe[1,2]

or

Safe[2,2]